ASSIGNMENT- 04

Bachelor of Computer Applications (Hons. AI & ML)
Semester- III

Subject: Database Management System

Topic: Relational Database vs. NoSQL Database

By
Name: Mohammed Sinan nm
Reg. No.: 2511021240024

Faculty Name: RENITA JOY
Faculty Signature:_____

Department of Computer Application
Alliance University
Chandapura - Anekal Main Road, Anekal
Bengaluru - 562 106
August 2026

RELATIONAL DATABASE VS. NOSQL DATABASE

1. Introduction

Databases are essential for storing, organizing, retrieving, and managing information. As applications grow, selecting the right database technology becomes important for performance, reliability, scalability, and cost.

Two major database approaches are relational databases and NoSQL databases. Relational databases organize information into tables containing rows and columns, with relationships between tables. NoSQL databases use different data models, including document, key-value, wide-column, and graph models, to support various types of information and workloads.

This report compares relational and NoSQL databases in terms of scalability, consistency, performance, and use cases. It also explains why organizations choose different database technologies according to their business and technical requirements.

2. Relational Databases

A relational database stores structured data in tables. Each table contains rows and columns, and tables are connected using keys such as primary keys and foreign keys. SQL (Structured Query Language) is commonly used to create, query, and manage relational data.

Examples: MySQL, PostgreSQL, Oracle Database, and Microsoft SQL Server.

Key Characteristics

- Data is organized into tables with a defined structure.
- Relationships between tables are established using primary keys and foreign keys.
- SQL supports querying, inserting, updating, and deleting data.
- Transactions commonly support ACID properties: Atomicity, Consistency, Isolation, and Durability.
- Constraints help maintain data integrity and enforce business rules.
- Relational databases are suitable for structured data and applications requiring reliable transactions.

3. NoSQL Databases

NoSQL databases are non-relational database systems designed to support data models and workloads that may not fit well into traditional relational tables. They include document, key-value, wide-column, and graph databases.

Examples: MongoDB, Redis, Apache Cassandra, and Neo4j.

Key Characteristics

- Data models can be flexible and may support changing or semi-structured data.
- Many NoSQL systems are designed to distribute data across multiple servers.
- They can provide efficient access when the database model matches the application's query patterns.
- Consistency guarantees vary depending on the database system.
- Some NoSQL databases provide strong consistency, while others support eventual or tunable consistency.
- Different NoSQL models are suitable for different applications.

4. Comparison Between Relational and NoSQL Databases

Factor	Relational Database	NoSQL Database
Data Model	Uses tables with rows and columns.	Uses document, key-value, wide-column, or graph models.
Schema	Usually has a predefined schema.	Often supports flexible or model-specific structures.
Scalability	Commonly scales vertically, although horizontal scaling is also possible.	Many systems are designed to scale horizontally across multiple servers.
Consistency	Commonly provides strong transactional consistency and ACID support.	Consistency varies by database and configuration.
Performance	Effective for complex queries, joins, and transactional workloads.	Can perform well for specific access patterns and distributed workloads.
Relationships	Supports relationships through keys and SQL joins.	Relationships may be embedded, duplicated, or handled through graph structures, depending on the model.
Transactions	Mature support for multi-row and multi-table transactions in many systems.	Transaction support varies between database systems.
Data Structure	Best suited to structured data and clearly defined relationships.	Useful for flexible, semi-structured, or specialized data models.

Use Cases	Banking, accounting, inventory, order processing, and booking systems.	Caching, content catalogs, distributed applications, event storage, and social graphs.
Examples	MySQL, PostgreSQL, Oracle Database, SQL Server.	MongoDB, Redis, Cassandra, Neo4j.

5. Scalability

Scalability refers to a database's ability to handle growth in data volume, number of users, or workload.

5.1 Relational Database Scalability

Relational databases are commonly scaled vertically by increasing the CPU, memory, or storage capacity of a server. They can also be scaled horizontally using replication, partitioning, sharding, or distributed database architectures.

However, horizontal scaling may require additional planning and coordination, especially when transactions involve multiple servers.

Example: An online banking application may use PostgreSQL or Oracle Database to manage accounts and transactions. Read replicas can be introduced to handle additional read requests.

5.2 NoSQL Database Scalability

Many NoSQL databases are designed to distribute data across multiple servers. This allows applications to handle growing datasets and large numbers of requests.

However, scalability depends on the particular database, data distribution strategy, consistency requirements, and workload.

Example: Apache Cassandra is designed for distributed data storage and high-volume workloads. It can be useful for applications that generate large amounts of event or time-series data.

6. Consistency and Reliability

Consistency refers to maintaining data accuracy and ensuring that database operations follow defined rules.

6.1 Relational Databases

Relational databases commonly support ACID transactions.

- **Atomicity:** A transaction is completed fully or not at all.
- **Consistency:** A transaction preserves defined database rules.
- **Isolation:** Concurrent transactions are controlled to prevent unwanted interference.

- **Durability:** Committed changes remain stored.

These properties make relational databases suitable for applications requiring accurate and reliable transactions.

Example: When money is transferred between two bank accounts, the debit and credit should be handled as one transaction. If the transaction fails, the system should not leave only one account updated.

6.2 NoSQL Databases

Consistency in NoSQL databases depends on the system and its configuration. Some provide strong consistency, while others allow replicas to become consistent over time.

Eventual consistency means that, if no new updates occur and the system continues operating normally, replicas are expected to converge to the same value.

This approach can be useful for distributed applications, but it may allow users to temporarily see outdated information.

Example: A social media application may tolerate a short delay before a profile update appears across all servers. However, financial transactions generally require stricter consistency guarantees.

7. Performance

Database performance depends on data structure, indexes, query patterns, transaction requirements, network conditions, and system configuration. Neither relational nor NoSQL databases are always faster; the best choice depends on the workload.

7.1 Relational Database Performance

Relational databases are effective for queries involving multiple related tables, joins, aggregations, and transactions. Indexes and query optimization can improve response time.

Example: An online retail system can combine Customers, Orders, OrderItems, and Payments to generate sales reports and customer purchase histories.

7.2 NoSQL Database Performance

NoSQL databases can provide high performance when their data models are designed around the application's access patterns.

For example:

- Document databases can retrieve related fields together.
- Key-value databases can provide fast lookups using keys.
- Wide-column databases can support large distributed workloads.
- Graph databases can efficiently explore relationships between connected entities.

Examples:

- Redis is commonly used for caching and fast key-based access.
- MongoDB can store flexible product catalog documents.
- Neo4j can represent relationships between users, friends, products, and recommendations.

The performance of each database depends on how well its structure matches the application's requirements.

8. Why Organizations Choose Different Database Technologies

Organizations select database technologies based on their data structure, consistency requirements, workload, expected growth, performance targets, cost, and technical expertise.

8.1 Need for Transaction Accuracy

Banks, payment services, and accounting systems often choose relational databases because they require reliable transactions, data integrity, and clearly defined relationships.

For example, a banking system must maintain accurate account balances and ensure that financial transactions are processed correctly.

8.2 Large-Scale Distributed Workloads

Applications that receive enormous volumes of requests or events across multiple regions may choose distributed NoSQL databases designed to scale across servers.

For example, a system collecting data from thousands of devices may use a distributed database to manage large volumes of incoming records.

8.3 Flexible or Changing Data

Content platforms and product catalogs may use document databases when records contain different fields or change frequently.

For example, an online shopping platform may store clothing sizes, electronic specifications, and furniture dimensions as different sets of attributes.

8.4 Fast Lookups and Caching

Applications requiring quick access to sessions, temporary values, counters, or cached results may use key-value databases such as Redis.

For example, an e-commerce website can cache frequently accessed product information to reduce the workload on its main database.

8.5 Complex Relationships

Social networks, recommendation systems, and fraud detection applications may benefit from graph databases because connections between entities are central to their queries.

For example, a recommendation system can analyze relationships between users, products, and interests to identify relevant suggestions.

8.6 Existing Systems, Skills, and Cost

Organizations also consider licensing costs, cloud services, staff expertise, backup and recovery requirements, monitoring, migration expenses, and compatibility with existing applications.

A technically suitable database may not be the best choice if the organization cannot operate or maintain it effectively.

9. Real-World Examples

Organization or Application Type	Suitable Database Technology	Reason
Banking and payment processing	PostgreSQL, Oracle Database, or SQL Server	Supports transactions, integrity constraints, and relationships between accounts and payments.
Online retail orders and inventory	MySQL or PostgreSQL	Manages structured order data, joins, stock control, and transactional updates.
Product catalogs with varied attributes	MongoDB or another document database	Supports product records with different attributes by category.
Caching and session storage	Redis	Provides fast key-based access to temporary data and cached results.
Large-scale distributed event storage	Apache Cassandra	Supports distributed storage and high-volume workloads.
Social connections and recommendations	Neo4j or another graph database	Represents relationships between users, products, and interests.

These are representative technology choices rather than claims that every named organization uses the listed database. Large organizations often use several database technologies for different parts of the same system.

10. Using Relational and NoSQL Databases Together

Organizations do not always have to choose between relational and NoSQL databases. They can use both technologies in the same application, an approach commonly called **polyglot persistence**.

For example, an online food delivery platform could use:

- **PostgreSQL or MySQL** to manage customers, orders, and payments.
- **Redis** to cache restaurant listings or store temporary session data.
- **MongoDB** to manage flexible menu information, if a document model suits the application's requirements.

Using multiple databases allows each technology to handle the tasks for which it is best suited.

However, this approach also increases system complexity, monitoring requirements, and operational costs. Organizations must carefully manage data synchronization, consistency, and failure recovery between systems.

11. Conclusion

Relational and NoSQL databases both play important roles in modern information systems. Relational databases provide structured tables, powerful SQL queries, relationships, and strong transactional support. These features make them suitable for banking, order processing, inventory management, and other applications requiring reliable transactions.

NoSQL databases provide different data models and, in many cases, flexible schemas and horizontal scaling for particular distributed workloads. Document, key-value, wide-column, and graph databases address different requirements, so NoSQL should not be treated as a single technology with identical characteristics.

Organizations choose database technologies by evaluating their data structure, consistency needs, workload, performance targets, scalability, cost, and technical expertise.

In some cases, one relational or NoSQL database is sufficient. In other cases, combining technologies is the most practical approach. Ultimately, the best database is the one that meets the application's requirements reliably, securely, and efficiently.